

Machine Learning Reviewer for the Midterm

CS0075 Machine Learning Algorithm, midterm lecture exam (all of Module 1 and Module 2). Built from the slides, including the picture slides, plus the verified formative questions. The blue boxes are memory aids I made (they are not from the slides) and the red boxes are the traps.

How To Use This Reviewer

The midterm covers **all of Module 1 and Module 2**. Parts: **(1) core concepts, (2) evaluation and complexity, (3) the algorithms, (4) traps, (5) code and imports**. Section 3.4 onward is the material SA1 did not cover. The cram sheet is the last page.

Part 1. Core Concepts and Definitions

What Machine Learning Is

- Machine learning is about **extracting knowledge from data**. It sits at the intersection of **statistics, artificial intelligence, and computer science**, and is also known as **predictive analytics** or **statistical learning**.
- Arthur Samuel, 1959**: giving computers the ability to **learn without being explicitly programmed**.
- ML is a discipline of AI that lets machines **automatically learn from data and past experiences**, identify patterns, and make **predictions** with **minimal human intervention**.

The Formal Definition (E, T, P)

A program learns from **experience E** with respect to a class of **tasks T** and **performance measure P** if its performance at T, as measured by P, **improves with experience E**.

Memory Aid

Experience, Task, Performance. Every Try Pays off: more experience, better score.

AI, ML, Deep Learning, and Data Science

- The slide diagrams nest them: **Artificial Intelligence contains Machine Learning, which contains Deep Learning**.
- Data Science** is collection, preparation, and analysis of data, leveraging AI/ML, research, industry expertise, and statistics to make business decisions.
- AI** is technology for machines to understand, interpret, learn, and make "intelligent" decisions, and includes ML among many other fields.
- ML** is algorithms that help machines improve through supervised, unsupervised, and reinforcement learning. It is a **subset of AI** and a **Data Science tool**.

Rule-Based Algorithm vs Machine Learning

	Traditional Rule-Based Algorithm	Machine Learning
What drives it	A condition (true goes to Code 1, false goes to Code 2)	Training data goes into the ML algorithm, which produces a model
How rules arise	Explicit programming ; the rules can be manually specified	Rules are automatically learned by machines
Best when	Rules are simple	Decision-making rules are complex or difficult to describe ; samples are used for training
Prediction	Not applicable	New data goes into the model, which outputs a prediction

Memory Aid

Rule-based: **Rules** in, **Answers** out. ML flips it: **Data** and answers in, the **Rules** out.
Fill-in-the-blank: "Rule-based is Condition. Machine learning is **Data**."

The scenario chart plots **rule complexity** (simple to complex) against **scale of the problem** (small to large):

	Small Scale	Large Scale
Complex rules	Manual rules	Machine learning algorithms
Simple rules	Simple problems	Rule-based algorithms

Why Machine Learning

- Computers operate **autonomously without explicit programming**; fed new data they learn, grow, and adapt.
- Performance **adaptively improves as the number of available samples increases**.
- Machines work **24/7**, need no breaks, cost much less to maintain than a human in the long run, and suit tasks that are **routine, repetitive, or tedious**.
- Automation also **mitigates risks caused by fatigue or inattention**. Slide examples: self-driving cars and robotic arms on production lines.
- Application areas on the slides: healthcare, automobile, transportation, manufacturing, e-commerce, insurance; medical diagnosis, image and speech recognition, fraud detection, recommendations, spam filtering, self-driving cars, traffic prediction, stock market trading.

Types of Machine Learning and the Workflow

The types-of-ML picture shows **four**: **Supervised, Unsupervised, Semi-Supervised, and Reinforcement Learning**. The "How does ML work" text lists three under Learning (supervised, unsupervised, reinforcement).

Phase	Steps
Phase 1: Learning	Training data, preprocessing (clean data, format data), learning ,

	testing (measure performance, test algorithm)
Phase 2: Prediction	New data goes into the trained model , which outputs predicted data

In the picture, the training data trains the ML algorithm, then **accuracy** is checked: **unacceptable** sends you back to train again, **acceptable** gives a **successful model**.

The Four-Stage ML Workflow (DataCamp Slide)

Stage	Steps
1. Project setup	Understand the business goals; choose the solution to your problem
2. Data preparation	Data collection, data cleaning , feature engineering , split the data
3. Modeling	Hyperparameter tuning, train your models , make predictions , assess model performance (a repeating cycle)
4. Deployment	Deploy the model, monitor model performance, improve your model

Memory Aid

Set up, Prepare, Model, Deploy: Some People Make Dinner.

Programming Languages and Python Tools

- The top programming languages for ML, in rank order: **1 Python, 2 R, 3 Java, 4 Julia, 5 Scala, 6 C++, 7 JavaScript, 8 Lisp, 9 Haskell, 10 Go.**
- Why Python:** easy-to-read syntax, extensive libraries and frameworks, strong community support, flexibility, compatibility with other languages, scalability and performance.

Name	What The Slide Says
Scikit-learn	The most prominent Python library for ML
NumPy	A fundamental package for scientific computing (mathematical functions)
SciPy	A collection of functions for scientific computing
Matplotlib	The primary scientific plotting library (data visualizations)
Seaborn	Data visualizations
Pandas	A library for data wrangling and analysis (data analysis and manipulation)
TensorFlow	Machine learning
Keras	Deep learning
PyTorch	Machine learning
Scrapy	Web crawling
SQLModel	Interacts with SQL databases
Anaconda	A Python distribution for large-scale data processing, predictive analysis, and scientific computing
Jupyter Notebook	An interactive environment for running code in the browser

Watch Out

Formative Q5 ("most popular libraries, choose all") keyed **PANDAS, SCIPY, NUMPY**. Anaconda is a **distribution** and Jupyter is an **interactive environment**, so they are not libraries. SQL is a query language.

Data Preprocessing

Real data has three quality problems: **incompleteness** (missing values or attributes), **noise** (incorrect records or exceptions), and **inconsistency** (inconsistent records). "**Without good data, there is no good model.**" Preprocessing (also called data preparation) is cleaning, altering, and reorganizing raw data before analysis.

A data scientist's time (pie chart): **60 percent data cleaning and organization**, 19 percent data set collection, 9 percent data mining, 4 percent algorithm improvements, 3 percent training set creation, 5 percent other.

Errors The Slide Marks In A Messy Table

Error	Example Meaning
Missing value	A blank cell
Invalid value	A value outside what is allowed (for example a letter in a gender column that expects M or F)
Invalid duplicate item	The same Id appears twice
Value in another column	A value that should be in a different column
Misspelling	For example "Ytali" for Italy
Incorrect format	A date written in a different format
Attribute dependency	Columns whose values contradict each other

Key Steps in Order

Data profiling, data cleansing, data reduction, data transformation, data enrichment, data validation.

Category	Techniques
Data Cleansing	Identify and sort out missing data , reduce noisy data , identify and remove duplicates
Feature Engineering	Feature scaling or normalization, data reduction, discretization, feature encoding

Outliers

An **outlier** is an observation that seems to be **distant from other observations** or, more specifically, one observation that **follows a different logic or generative process** than the other observations. **A single outlier can throw off a regression estimate** (the regression line gets pulled toward it).

Exploratory Data Analysis (EDA)

EDA understands the main characteristics of the data, spots patterns and **anomalies**, tests a hypothesis, or checks assumptions, often with visualization.

Activity	What It Does
Visualization	Plots and charts of distributions and relationships. Bar plots and histograms visualize the count of values
Summary Statistics	Mean, median, variance, standard deviation
Outlier Detection	Identifying unusual data points
Correlation Analysis	Examining relationships between variables
Hypothesis Testing	Testing initial assumptions about the data

Memory Aid

The point is the **outlier**; the process is **outlier detection**. "Identifies unusual data points" means the process.

The Titanic Example Used In The Slides

- **Titanic Survival Data Set:** 1309 passengers and whether they survived; the goal is to **predict survival from the attributes**. Tabular data can have **different data types in one table**.
- Types: **Survived** is integer binary (the **label**); **Pclass** is integer categorical or ordinal; **Name** and **Ticket** are strings; **Sex** and **Embarked** are string categorical; **Age** and **Fare** are continuous; **SibSp** and **Parch** are non-negative integers.
- **Feature selection:** dropping columns that are not useful (for example Name, Ticket, Cabin).
- **Missing values:** Age had **177** missing values and Embarked had **2**. Two options: **remove rows** with missing values, or **imputation** (filling in missing values by estimating them).
- **Mean imputation** (for a numeric column) and **most-frequent imputation** (mode, for a categorical column). Other imputers: median, iterative, kNN.
- **Column transformation** before training: **standard scaling** for numerical data (Age, Fare), **ordinal encoding** for categorical data (Pclass, Sex, Embarked); other features are retained.

Discrete vs Continuous Data

Type	Meaning	Example
Discrete	Countable in whole units; you cannot have a fraction of it	Number of students in a class
Continuous	Lies on an unbroken scale	Height, weight, temperature, Age, Fare

Memory Aid

Discrete = Digits you can count. Continuous = Can slide anywhere on the scale.

Model Deployment

Deployment means taking a trained model and making it available in real applications. The stages are **Training, Validation, Deployment, Monitoring** (models can **drift** as real-world data evolves). Deployment means defining real-time data handling, storage, predictions, APIs and tools, hardware (cloud or on-prem), and a pipeline for continuous training. Best practices: **(1) choose the right infrastructure, (2) effective versioning and tracking, (3) robust testing and validation, (4) implement monitoring and alerting.**

Ethics and Privacy in Data Science

Five Principles	Meaning
Ownership	An individual owns their personal information; collecting it without consent is unethical
Transparency	Data subjects have a right to know how it is collected, stored, and used
Privacy	Protect personally identifiable information (PII): full name, birthdate, phone number, bank account number, street address
Intention	Ask why you need the data and what you will gain; malicious intent is unethical
Outcomes	Even good intentions can cause harm; unintended harm to a group is disparate impact

Memory Aid

Own it, Tell them, Protect it, Intend well, check Outcomes: **Our Team Protects Individuals' Ownership.**

Topic	What The Slides Say
Three new rules of data	Trust over transactions (consent); Insight over identity (re-think transferring PII); Flows over silos (teams share the flow of insights)
Regulations shown	GDPR (General Data Protection Regulation) and the National Privacy Commission (Philippines)
Data subject rights	Right to be informed , to damages , to access , to erasure or blocking , to file a complaint , to object , to rectify , and to data portability
Data governance	A set of principles and processes for data collection, management, and use, so data is accurate, consistent, and available while protecting privacy and security. A framework describes how to do what governance says the organization needs to do
Pillars of data governance	Ownership and accountability, data quality, data protection and safety, data use and availability, data management (built on people, processes, and technology)
10 questions before AI in public decisions	Objective, use, impacts, assumptions, data, inputs, mitigation, ethics, oversight, evaluation

Part 2. Evaluation, Generalization and Complexity

Supervised Learning Basics

- **Labeled training data** has both the **input** (data features) and the **correct output** (the label or goal). Each example is a pair: an input object (feature vector) and a desired output value.
- The primary objective is to **map the input variable to the output variable**: $y = f(x)$. In the slide example, weather, temperature, and wind speed are the input x and "Enjoy Sports" (yes or no) is the output y .
- Two phases: **training and validation**, followed by **prediction**.

	Classification	Regression
Target	Categorical variable	Continuous variable
Predicts	One of the possible class labels	Consecutive numbers (real numbers) , per the slide
Examples	Spam or not spam, COVID positive or negative on a chest X-ray, handwritten digit recognition	Stock price forecast, weather prediction, annual income forecast, population forecast
Subtypes	Binary (two classes) and multiple (three or more)	Simple linear, multivariate
Algorithms (slide)	Random Forest, Decision Tree, Logistic Regression, Support Vector Machine	Simple Linear Regression, Multivariate Regression, Decision Tree, Lasso Regression

Watch Out

Formative Q17. The slide says regression "**predicts consecutive numbers (real numbers)**". The graded formative key marked "Regression predicts consecutive numbers" as **False** (regression predicts **continuous real values**, not a counting sequence like 1, 2, 3). Study rule: **regression = continuous real numbers**. If a true or false item says "consecutive numbers" with no mention of real numbers, go with the **formative key (False)**, because your teacher wrote both the quiz and the exam. If the item says "real numbers", it is True.

Generalization, Overfitting, Underfitting

Term	What The Slides Say	Train	Test
Generalization	The model makes accurate predictions on new, unseen data with the same characteristics as the training set. We want to generalize as accurately as possible .	Good	Good
Overfitting	The model fits the particularities of the training set (its noise and outliers) too closely; it has memorized the training data	Very good	Poor

	rather than learned the pattern.		
Underfitting	The model is too simple to capture the variability in the data and does badly even on the training set .	Poor	Poor

Memory Aid

Under = Underbuilt (bad everywhere). **Over = Overthinks** (aces training, flops on new data).

Model Complexity and the Complexity Curve

- The curve plots **accuracy** against **model complexity**. **Training accuracy keeps rising** as complexity rises. **Generalization (test) accuracy rises, peaks at the sweet spot, then falls.**
- **Left side = underfitting** (too simple). **Right side = overfitting** (too complex). The **sweet spot** in between gives the **best generalization**.
- The more complex the model, the better it predicts the **training** data. Too complex, and it focuses on each individual point and **does not generalize**.
- The larger the **variety** of data points, the more complex a model you can use without overfitting. **Duplicating the same points or collecting very similar data does not help.**
"Never underestimate the power of **more data**."

Complexity	Fit Problem	Error Type (Slide Definition)
Too low (too simple)	Underfitting	Bias = error from wrong or too-simple assumptions in the algorithm
Too high (too flexible)	Overfitting	Variance = error from sensitivity to noise and fluctuations in the training data

The **dartboard** picture: **low bias and low variance = good model** (darts tightly clustered on the bullseye). High bias means the cluster is off the bullseye; high variance means the darts are spread out.

Watch Out

"Low complexity is overfitting" is **False**. Low complexity is **underfitting**.

The boat example: a rule that is 100 percent accurate on 12 rows is not trustworthy. If 10,000 more rows obeyed the same rule, you would believe it. More data supports a more complex model.

R-Squared (R², Coefficient of Determination)

$R^2 = 1 - [\text{sum of } (y - \hat{y}) \text{ squared}] / [\text{sum of } (y - \bar{y}) \text{ squared}]$, where **y** is the target value, **y-bar** the average value, and **y-hat** the predicted value.

Situation	R ²
Perfect prediction (prediction equals target, numerator equals	1

zero)	
Predicting the average (numerator equals denominator): a constant model that predicts the mean of <code>y_train</code>	0
Worse than predicting the average	Negative

Memory Aid

R2 is a **report card against guessing the average**: 1 is perfect, 0 is no better than the average, **below 0 is worse than the average**.

Watch Out

The slide also says R2 "yields a score between 0 and 1", but the next slide says it **can be negative**. "Predicting worse than the average can give a negative number" is **True** (formative Q11). For regressors, scikit-learn's **score()** returns **R2**; for classifiers it returns **accuracy**.

Part 3. The Algorithms

3.1 k-Nearest Neighbors (k-NN)

- Arguably the **simplest** ML algorithm. **Building the model is only storing the training dataset**.
- To predict, it finds the **closest training points** (the nearest neighbors).
- **Classification**: with $k = 1$, the prediction is the label of the single nearest point. With $k > 1$ it uses **voting**: the **majority class** among the k neighbors wins. Works for any number of classes.
- **Regression**: with $k = 1$, the prediction is the target of the nearest neighbor. With $k > 1$ the prediction is the **average (mean)** of the neighbors.
- In scikit-learn: **KNeighborsClassifier** and **KNeighborsRegressor**; train with **fit**, predict with **predict**, evaluate with **score**.

	Small k (1 neighbor)	Large k (9 neighbors)
Decision boundary	Jagged; predictions go through every point (unsteady)	Smoother ; does not fit the training data as well
Model complexity	More complex	Less complex
Bias and variance	Bias goes down , variance goes up	Bias goes up , variance goes down
Risk	Overfitting	Underfitting

On the **accuracy against `n_neighbors`** plot, **training accuracy falls** as `n_neighbors` grows, test accuracy peaks around 6, and **the best performance is somewhere in the middle** (the optimal point). The left side is overfitting and the right side is underfitting.

Memory Aid

Small k = Small crowd = Super complex = Overfit. Big k = Big crowd = Blurry average = Underfit.

Watch Out

"A small k makes the model less complex" is **False** (formative Q9). "In k-NN, low model complexity is overfitting" is **False** (Q10): low complexity is underfitting.

Important Parameters	Strengths	Weaknesses
Number of neighbors and the distance measure (Euclidean) by default). Three or five neighbors usually works well, but adjust it.	Easy to understand; works well without special adjustments ; suitable as a first model	Slow prediction when features or samples are many, so preprocessing matters; does not work well with sparse datasets

3.2 Linear Models

- Linear models make a prediction using a **linear function of the input features**: a **best-fit line**.
- They assume a **linear relationship** between the outcome and each predictor. Examples: boiling point vs altitude, advertising spend vs revenue, fertilizer vs crop yield, athlete performance vs training.
- Types: **Linear Regression** (regression, numeric output) and **Logistic Regression** (classification).

The Prediction Formula

$$\hat{y} = w[0]*x[0] + w[1]*x[1] + \dots + w[p]*x[p] + b$$

Symbol	Meaning
x[0] to x[p]	The features (p + 1 of them)
w[0] to w[p]	The weights or coefficients (the slopes)
b	The intercept , also called the offset or bias
y-hat	The model's prediction

Memory Aid

w = **W**eight = slope. **b** = **B**ias = intercept = offset. All three of intercept, offset, and bias point at **b**.

In scikit-learn the weights are stored in **coef_** and the intercept in **intercept_**. "The offset parameter is also called the intercept" is **True** (formative Q12).

3.3 Linear Regression (Ordinary Least Squares)

- Linear regression = ordinary least squares (OLS)**: the simplest and most classic linear method for regression.

- It finds **w** and **b** that **minimize the mean squared error** between predictions and the true targets on the **training set**.
- The **mean squared error** is the sum of squared differences between the predictions and the true values (as the slide words it).

Example	Train R2 vs Test R2	Diagnosis
Wave dataset (one feature)	About 0.66 , and the train and test scores are very close	Likely underfitting , not overfitting
Extended Boston Housing (506 samples, 104 or 105 features)	Very accurate on train , much worse on test	Overfitting . Too many features inflate the weights. Try a model that controls complexity : Ridge

The extended Boston data is the **13 original features plus the 91 two-feature products** (the slide's shape is (506, 104); the slide text elsewhere says 105 features). Including derived features like these is **feature engineering**.

Memory Aid

Close scores, mediocre = underfit. Big gap, high train = overfit.

SA1 stopped at the end of 3.3. Everything from here is the extra material for the midterm.

3.4 Ridge Regression

- A linear model for regression with the **same prediction formula as OLS**.
- The coefficients are also chosen to be **as small as possible** (close to zero, **but not zero**), so each feature has as little effect as it can while still predicting well.
- That constraint is **regularization: explicitly restricting a model to avoid overfitting**. Ridge uses **L2 regularization** (the **L2 norm squared**).
- **Cost function: $MSE + \alpha * (\text{sum of } w \text{ squared})$** . Larger alpha, larger penalty, smaller w. Default **alpha = 1.0**.

	Ridge vs LinearRegression (Boston)
Training score	Ridge is lower (0.89 in the slide)
Test score	Ridge is higher (0.75 in the slide); less overfitting because Ridge is a more restricted model
More training data	The gap closes; with enough data regularization matters less . OLS at tiny sizes could even score R2 below 0

Memory Aid

Alpha up, All weights shrink toward zero. A bigger alpha is a **bigger fine**.

Watch Out

"In Ridge, a larger alpha makes the penalty lesser" is **False** (formative Q13). Ridge weights **approach zero but never reach it**.

3.5 Lasso

- An alternative to Ridge. It also keeps coefficients near zero, but with **L1 regularization** (the L1-norm, a diamond shape in the slide picture).
- **Cost function: $MSE + \alpha * (\text{sum of the absolute values of } w)$** . Consequence: **some coefficients become exactly zero**, so some features are **ignored entirely**.
- Alpha = 1.0 (default) was too much regularization: **underfitting**, using only **4 of the 105 features**. **Lowering alpha** used **33 of 105 features** and did slightly better than Ridge.
- Alpha = 0.0001 was too low: train 0.95, test 0.64, **96 features used**, so it behaves **like linear regression**.

	Ridge	Lasso
Penalty	L2 (squared weights)	L1 (absolute weights)
Weights	Approach zero, never exactly zero	Can be exactly zero
Prefer it when	Generally preferred (L2 over L1)	Only some of many features matter , or you want a model that is easy to analyze and understand

Memory Aid

L1 = Leaves out features (zeros). **L2 = Loves them all a little** (shrinks, keeps).

3.6 Logistic Regression and LinearSVC

- Linear models for **classification**. For binary classification, the prediction is positive (+1) if the formula is **greater than 0** and negative (-1) if **less than 0**; $\hat{y}$ is the **decision boundary**.
- The boundary is a **line** (1 feature), a **plane** (2 features), and a **hyperplane** (3 or more).
- Classification in linear models depends on how well the weights and intercept fit the training data, the **cost or loss function**, and **regularization**.
- Logistic regression** is a classification algorithm despite its name. It gives a **probability between 0 and 1** using the **logistic (sigmoid) function**, and a **threshold** decides the class.
- The two common linear classifiers: **LogisticRegression** and **linear SVM (LinearSVC)**.
- Regularization parameter **C**: **higher C = less regularization** (fits the training set as well as it can); **low C** pushes **w toward zero**. Default **C = 1** with **L2**. On the cancer data, **C = 1** gave 94 percent on both train and test (likely underfitting); **L1** gives a more **interpretable** model using fewer features.

Watch Out

C works **opposite** to **alpha**: **high C = weak regularization**, **high alpha = strong regularization**.

Memory Aid

C = Control is loose when high. **Alpha = Accuse** harder when high.

3.7 Naive Bayes

- A family of classifiers quite similar to linear models; **training is faster** than a linear classifier, with slightly lower generalization.
- They are efficient because they learn parameters by looking at **each feature individually** and collecting **simple per-class statistics** from each feature (formative Q15: **True**).
- Alpha** controls model complexity by smoothing statistics. **Larger alpha decreases complexity** but does not change performance much.
- Works well with **sparse, high-dimensional** data and is **not parameter sensitive**. Training and prediction are fast and easy to understand.

Classifier	Data
GaussianNB	Continuous data
BernoulliNB	Binary data, text data
MultinomialNB	Integer count data, text data

Memory Aid

Gaussian = Grades on a scale (continuous). Bernoulli = Binary. Multinomial = Many counts.

3.8 Decision Trees

- Used for **classification and regression**. They learn a **hierarchy of if/else questions** leading to a decision.
- In the animal tree picture: the top node is the **root node** ("Has feathers?"), the questions are the **nodes** (characteristics: "Can fly?", "Has fins?"), the arrows are **edges** (true or false), and the answers (Hawk, Penguin, Dolphin, Bear) are **terminal nodes** (leaves).
- The questions are called **tests** (not the test set). The algorithm picks the **most informative** test about the target. For continuous data a test looks like "Is feature greater than a?". The root represents the whole dataset.
- A prediction finds the region the point falls in by going left or right from the root, then returns the **majority target** (or the single target in a pure leaf).
- scikit-learn: **DecisionTreeClassifier** and **DecisionTreeRegressor**. Visualize with **export_graphviz** (a **.dot** file). **Feature importances** show what the tree used.
- Tree regression is **not affected by feature scale**, but it **cannot extrapolate**: on the RAM price data its prediction stays **flat** after the training range, while the linear model keeps following the trend.

3.9 Ensembles: Random Forest and Gradient Boosting

Ensembles combine multiple models into a more powerful one. Both below use **decision trees** as building blocks.

	Random Forest	Gradient Boosting
Idea	Reduces overfitting by averaging many trees; injects randomness	Each new tree compensates for the error of the previous tree
Trees	Built independently ; each from a bootstrap sample (draw with replacement; about one third of points missing)	Shallow trees, depth 5 or less ; pre-pruning is applied, not randomness
Prediction	Regression: average of values . Classification: average of predicted probabilities	Regression: least squares loss. Classification: logistic loss. Uses gradient descent
Key parameters	n_estimators , max_features	n_estimators , learning_rate (default 0.1), max_depth
Strengths	Widely used; excellent performance; little tuning; no scaling needed	Slightly better than random forest ; no scaling needed
Weaknesses	Hard to analyze; poor on large sparse data; more memory and slower than linear models	Sensitive to parameters ; longer training; poor on sparse high-dimensional data

Memory Aid

Forest = many independent trees VOTE (average). Boosting = trees take TURNS fixing the last one's mistakes.

On the Breast Cancer data: random forest gave **97 percent** with no tuning; gradient boosting at defaults hit **100 percent on training**, a sign of **overfitting**, fixed by **limiting depth** or **lowering the learning rate**.

3.10 Kernelized Support Vector Machines

- An extension of linear SVM for **nonlinear** problems that a hyperplane cannot solve. **SVC** for classification, **SVR** for regression.
- **Support vectors** are the points that define the decision boundary. The **kernel trick** gives complex boundaries even with few features.
- **gamma** controls the width of the **Gaussian (RBF) kernel**: **small gamma = large radius = many points count as close = simpler**.
- **C** limits each point's influence: **small C = very restricted model**.
- SVMs are **sensitive to feature scale**. Rescale to **0 to 1 with MinMaxScaler**. On the Breast Cancer data this fixed the overfitting, then **increasing C** reached **97.2 percent**.
- Weakness: **slow with many samples (more than about 100,000)**; sensitive to preprocessing and parameters. Parameters: **C**, **gamma** (RBF), coef (sigmoid), degree (polynomial).

Part 4. High-Yield Traps

These are the twists the formatives used, plus the slide details most likely to be tested. Most are a true idea with **one word flipped**.

Trap	Wrong Idea	Right Idea
Discrete vs continuous	Number of students is continuous	Discrete : countable whole units
Consecutive vs continuous	Regression predicts counting numbers	Regression = continuous real numbers . Formative key: "consecutive numbers" is False
Small k	Small k makes the model less complex	Small k = more complex = overfitting. False .
Low complexity	Low complexity is overfitting	Low complexity is underfitting . False .
Rule-based vs ML	ML runs on conditions	Rule-based = condition ; ML = data
Types of ML	There are three types	The picture shows four : supervised, unsupervised, semi-supervised , reinforcement
Libraries	Anaconda or Jupyter is a library	Libraries: pandas, SciPy, NumPy , etc. Anaconda = distribution; Jupyter

		= environment
Outlier word	Answering the process for the point	Point = outlier ; process = outlier detection
Missing values	Only one way to handle them	Remove rows or impute (mean for numeric, most frequent for categorical)
Python's rank	Java is the top ML language	Python is 1 , R is 2, Java is 3
R2 sign	R2 can never be negative	Worse than the mean gives negative R2. True.
Offset	Offset is not the intercept	Offset = intercept = bias (b). True.
score()	score() is always accuracy	Classifier = accuracy; regressor = R2
Ridge alpha	Larger alpha makes the penalty lesser	Larger alpha = bigger penalty. False.
Alpha vs C	High C means strong regularization	High C = weak regularization; high alpha = strong
Ridge vs Lasso zeros	Ridge zeroes out features	Lasso (L1) hits exact zero; Ridge (L2) only approaches it
Naive Bayes types	GaussianNB for counts	Counts = Multinomial ; continuous = Gaussian ; binary = Bernoulli
Logistic Regression	A regression algorithm	A classification algorithm despite its name
Trees and scale	Trees need scaled features	Trees and forests do not need scaling ; SVM and k-NN do
Forest vs boosting	Both build trees independently	Forest = independent trees averaged; boosting = trees built in turn , each fixing the last

Part 5. Code and Imports

Reported exam format: **fill-in-the-blank imports** (like import ____ as np) and **two 7-point code items** where you complete missing code taken from the module. Everything in this part is copied from the prof's M2.ipynb, the M1 preprocessing notebook, and the code shown in the M2 picture slides (read slide by slide). The slides name the k-NN model clf and the notebook names it knn, so know both.

Imports From The Module

What	Import line
NumPy	import numpy as np
pandas	import pandas as pd
Matplotlib	import matplotlib.pyplot as plt
mglearn	import mglearn
Legend patches	import matplotlib.patches as mpatches
Iris data	from sklearn.datasets import load_iris

Breast cancer data	from sklearn.datasets import load_breast_cancer
Train/test split	from sklearn.model_selection import train_test_split
k-NN classifier	from sklearn.neighbors import KNeighborsClassifier
Accuracy	from sklearn.metrics import accuracy_score
Forge / wave data	import mglearn, then mglearn.datasets.make_forge() or make_wave(n_samples=40)
Moons data	from sklearn.datasets import make_moons
California housing	from sklearn.datasets import fetch_california_housing
k-NN regressor	from sklearn.neighbors import KNeighborsRegressor
Linear regression	from sklearn.linear_model import LinearRegression
Ridge	from sklearn.linear_model import Ridge
Lasso	from sklearn.linear_model import Lasso
Logistic regression	from sklearn.linear_model import LogisticRegression
Linear SVM	from sklearn.svm import LinearSVC
Kernel SVM	from sklearn.svm import SVC
Decision tree	from sklearn.tree import DecisionTreeClassifier (regression: DecisionTreeRegressor)
Export a tree	from sklearn.tree import export_graphviz, then import graphviz
Random forest	from sklearn.ensemble import RandomForestClassifier
Gradient boosting	from sklearn.ensemble import GradientBoostingClassifier
Missing-value imputer	from sklearn.impute import SimpleImputer
Z-score scaling	from sklearn.preprocessing import StandardScaler
0 to 1 scaling	from sklearn.preprocessing import MinMaxScaler

Memory Aid

Package = what it does: **datasets**, **model_selection** (splitting), **neighbors**, **impute**, **preprocessing** (scalers), **metrics**. The class name is the one with capitals.

Naive Bayes shows **no code** on the slides. They only name the three classes: **GaussianNB** (continuous), **BernoulliNB** (binary), **MultinomialNB** (counts). Their import path, sklearn.naive_bayes, is standard scikit-learn and is not shown on the slides.

Note: the slide code assumes imports it does not always show (train_test_split, plt, np, LinearRegression), so know those import lines on your own.

Code 1. k-NN On Iris (Notebook)

Load, split, build, train, predict, score

```
from sklearn.datasets import load_iris
iris_dataset = load_iris()

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    iris_dataset['data'], iris_dataset['target'], random_state=0)

from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=3)
```

```

knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)
print("Test set score: {:.2f}".format(knn.score(X_test, y_test)))
print("Test set score: {:.2f}".format(np.mean(y_pred == y_test)))    # same number
by hand

X_new = np.array([[40, 80, 70, 30]])
prediction = knn.predict(X_new)
print("Predicted target name:", iris_dataset['target_names'][prediction])

```

- Order to remember: **load, split, instantiate with n_neighbors, fit, predict, score.**
- fit(X_train, y_train) trains. predict(X_test) gives labels. score(X_test, y_test) gives accuracy.
- X_new must be **2-D**, so it is a list inside a list: np.array([[40, 80, 70, 30]]).
- Look at the data with iris_dataset.keys(), ['target_names'], ['feature_names'], ['data'].shape, X_train.shape.

Code 2. k-NN On Breast Cancer, With The Complexity Curve

DataFrame, stratified split, loop over k

```

from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer()

df = pd.DataFrame(data=cancer.data, columns=cancer.feature_names)
df['target'] = cancer.target

X = df.drop(columns='target')    # features
y = df['target']                 # target
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y,
random_state=66)

training_accuracy = []
test_accuracy = []
neighbors_settings = range(1, 10)
for n_neighbors in neighbors_settings:
    knn = KNeighborsClassifier(n_neighbors=n_neighbors)
    knn.fit(X_train, y_train)
    training_accuracy.append(knn.score(X_train, y_train))
    test_accuracy.append(knn.score(X_test, y_test))
plt.plot(neighbors_settings, training_accuracy, label="training accuracy")
plt.plot(neighbors_settings, test_accuracy, label="test accuracy")
plt.ylabel("Accuracy")
plt.xlabel("n_neighbors")
plt.legend()

knn = KNeighborsClassifier(n_neighbors=6)    # k picked from the curve
knn.fit(X_train, y_train)

```

- stratify=y keeps the **class proportions** the same in train and test. random_state makes the split repeatable.

- Cancer data: **569 samples, 212 malignant and 357 benign**, 30 features. Iris used random_state=0 and n_neighbors=3; cancer used random_state=66 and n_neighbors=6.
- Pattern: empty lists, a for loop over k, .append(knn.score(...)), then plt.plot both lists against k.

Code 3. Module 1 Preprocessing (Also On SA1)

Imputation and scaling

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer

df = pd.read_csv('diabetes_NaN_age 3.csv')

imputer_mean = SimpleImputer(strategy='mean', missing_values=np.nan)
df['Age'] = imputer_mean.fit_transform(df[['Age']]).ravel()

# the same fix with plain pandas
age_mean = df['Age'].mean()
df.loc[df['Age'].isna(), 'Age'] = age_mean

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
numeric_cols = df.select_dtypes(include='number').columns
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
```

- strategy can be 'mean', 'median', 'most_frequent', or 'constant'. Missing values are marked np.nan.
- fit_transform wants a **2-D** input, so it is df[['Age']] with **double brackets**; .ravel() flattens the result back to one column.
- pd.isnull(value) or .isna() finds missing values; int(value) inside try/except ValueError finds invalid integers.
- **StandardScaler** = mean 0, standard deviation 1. **MinMaxScaler** = squeezes to 0 to 1.
- select_dtypes(include='number').columns picks every numeric column.

Watch Out

Fill-in-the-blank traps: train_test_split returns **four** things in the order **X_train, X_test, y_train, y_test**. fit takes **train** data; score takes **test** data. n_neighbors is the k parameter. Double brackets [['Age']] give a table, single brackets give one column.

Slide Code (M2 Picture Slides)

Everything below is code printed in the M2 slide images, read slide by slide. Variable names are the slides' own.

Code 4. k-NN Classifier, Slide Version (clf)

Forge data, k = 3

```
from sklearn.model_selection import train_test_split
X, y = mglearn.datasets.make_forge()
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)

from sklearn.neighbors import KNeighborsClassifier
clf = KNeighborsClassifier(n_neighbors=3)
clf.fit(X_train, y_train)

print("Test set predictions: {}".format(clf.predict(X_test)))
print("Test set accuracy: {:.2f}".format(clf.score(X_test, y_test)))
```

Decision boundaries for k = 1, 3, 9 (loop lines)

```
for n_neighbors, ax in zip([1, 3, 9], axes):
    clf = KNeighborsClassifier(n_neighbors=n_neighbors).fit(X, y)
    mglearn.plots.plot_2d_separator(clf, X, fill=True, eps=0.5, ax=ax, alpha=.4)
    ax.set_title("{} neighbor(s)".format(n_neighbors))
```

Complexity curve on breast cancer (slide uses range(1, 11))

```
from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, stratify=cancer.target, random_state=66)

training_accuracy = []
test_accuracy = []
neighbors_settings = range(1, 11)

for n_neighbors in neighbors_settings:
    clf = KNeighborsClassifier(n_neighbors=n_neighbors)
    clf.fit(X_train, y_train)
    training_accuracy.append(clf.score(X_train, y_train))
    test_accuracy.append(clf.score(X_test, y_test))

plt.plot(neighbors_settings, training_accuracy, label="training accuracy")
plt.plot(neighbors_settings, test_accuracy, label="test accuracy")
plt.ylabel("Accuracy")
plt.xlabel("n_neighbors")
plt.legend()
```

Code 5. k-NN Regressor

Wave data, k = 3, score is R squared

```
from sklearn.neighbors import KNeighborsRegressor
```

```

X, y = mglearn.datasets.make_wave(n_samples=40)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)

# Create model object
reg = KNeighborsRegressor(n_neighbors=3)

# Model training
reg.fit(X_train, y_train)

print("Test set predictions:\n{}".format(reg.predict(X_test)))
print("Test set R^2: {:.2f}".format(reg.score(X_test, y_test)))

```

Code 6. Linear Regression, Ridge, Lasso

OLS on wave data

```

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import mglearn

X, y = mglearn.datasets.make_wave(n_samples=60)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

lr = LinearRegression().fit(X_train, y_train)

print("lr.coef_: {}".format(lr.coef_))
print("lr.intercept_: {}".format(lr.intercept_))
print("Training set score: {:.2f}".format(lr.score(X_train, y_train)))
print("Test set score: {:.2f}".format(lr.score(X_test, y_test)))

```

Ridge and Lasso (extended Boston Housing)

```

from sklearn.linear_model import Ridge
ridge = Ridge().fit(X_train, y_train)
print("Training set score: {:.2f}".format(ridge.score(X_train, y_train)))
print("Test set score: {:.2f}".format(ridge.score(X_test, y_test)))

from sklearn.linear_model import Lasso
import numpy as np
lasso = Lasso().fit(X_train, y_train)
lasso001 = Lasso(alpha=0.01, max_iter=100000).fit(X_train, y_train)
lasso00001 = Lasso(alpha=0.0001, max_iter=100000).fit(X_train, y_train)

plt.plot(ridge.coef_, 's', label="Ridge alpha=1")
plt.plot(lasso.coef_, 's', label="Lasso alpha=1")

```

- Ridge default **alpha=1.0**. The slides also try **alpha=10** (high penalty) and **alpha=0.1** (low penalty).

- Lasso default **alpha=1.0**, **max_iter=1000** underfits (**4 of 105 features** used). **alpha=0.01** with **max_iter=100000** works well. **alpha=0.0001** overfits (**96 of 105 features**, close to plain OLS).
- **max_iter** has to go up when alpha goes down, otherwise Lasso warns it did not converge.

Code 7. Logistic Regression and LinearSVC

Two linear classifiers, then logistic regression on breast cancer

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
import matplotlib.pyplot as plt
import mglearn

for model, ax in zip([LinearSVC(), LogisticRegression()], axes):
    clf = model.fit(X, y)
    mglearn.plots.plot_2d_separator(clf, X, fill=False, eps=0.5, ax=ax, alpha=.7)

cancer = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, stratify=cancer.target, random_state=42)
logreg = LogisticRegression().fit(X_train, y_train)

logreg100 = LogisticRegression(C=100).fit(X_train, y_train)
logreg001 = LogisticRegression(C=0.01).fit(X_train, y_train)

lr_l1 = LogisticRegression(C=C, penalty="l1", solver='liblinear').fit(X_train,
y_train)
print("Test accuracy of l1 logreg with C={:.3f}: {:.2f}".format(
    C, lr_l1.score(X_test, y_test)))
```

- Default **C=1.0** with an **L2** penalty. Default scores: training **0.948**, test **0.944**. **C=100** = weaker regularization; **C=0.01** = stronger.
- **penalty="l1"** needs **solver='liblinear'** and gives sparse coefficients (an easier model to read).

Code 8. Trees, Random Forest, Gradient Boosting

Decision tree on breast cancer

```
from sklearn.tree import DecisionTreeClassifier
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, stratify=cancer.target, random_state=42)
tree = DecisionTreeClassifier(random_state=0)
tree.fit(X_train, y_train)
print("Accuracy on training set: {:.3f}".format(tree.score(X_train, y_train)))
print("Accuracy on test set: {:.3f}".format(tree.score(X_test, y_test)))

# pre-pruning
tree = DecisionTreeClassifier(max_depth=4, random_state=0)
tree.fit(X_train, y_train)
```

Export and view the tree

```
from sklearn.tree import export_graphviz
export_graphviz(tree, out_file="tree.dot", class_names=["malignant", "benign"],
                feature_names=cancer.feature_names, impurity=False, filled=True)

import graphviz

with open("tree.dot") as f:
    dot_graph = f.read()
graphviz.Source(dot_graph)
```

Tree regressor vs linear regression (RAM prices)

```
from sklearn.tree import DecisionTreeRegressor
data_train = ram_prices[ram_prices.date < 2000]
data_test = ram_prices[ram_prices.date >= 2000]

X_train = data_train.date[:, np.newaxis]
y_train = np.log(data_train.price)    # log-transform

tree = DecisionTreeRegressor().fit(X_train, y_train)
linear_reg = LinearRegression().fit(X_train, y_train)

X_all = ram_prices.date[:, np.newaxis]
pred_tree = tree.predict(X_all)
pred_lr = linear_reg.predict(X_all)

price_tree = np.exp(pred_tree)        # undo log-transform
price_lr = np.exp(pred_lr)
```

Random forest

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=100, noise=0.25, random_state=3)
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y,
                                                    random_state=42)
forest = RandomForestClassifier(n_estimators=5, random_state=2)
forest.fit(X_train, y_train)

# 100 trees on breast cancer
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, random_state=0)
forest = RandomForestClassifier(n_estimators=100, random_state=0)
forest.fit(X_train, y_train)
print("Accuracy on training set: {:.3f}".format(forest.score(X_train, y_train)))
print("Accuracy on test set: {:.3f}".format(forest.score(X_test, y_test)))
```

Gradient boosting

```
from sklearn.ensemble import GradientBoostingClassifier

X_train, X_test, y_train, y_test = train_test_split(
```

```

cancer.data, cancer.target, random_state=0)

gbrt = GradientBoostingClassifier(random_state=0)
gbrt.fit(X_train, y_train)
print("Accuracy on training set: {:.3f}".format(gbrt.score(X_train, y_train)))
print("Accuracy on test set: {:.3f}".format(gbrt.score(X_test, y_test)))

gbrt = GradientBoostingClassifier(random_state=0, max_depth=1)      # stronger
pre-pruning
gbrt = GradientBoostingClassifier(random_state=0, learning_rate=0.01)  # lower
learning rate

```

- Tree: **max_depth=4** is the pre-pruning fix for the fully grown tree (full tree: train **1.000**, test **0.937**; max_depth=4: train **0.988**, test **0.951**). **random_state=0** keeps the tree repeatable.
- Forest: **n_estimators** = number of trees (5 on moons, 100 on breast cancer); **max_features** is the other key parameter.
- Gradient boosting defaults: **n_estimators=100**, **max_depth=3**, **learning_rate=0.1**. Overfit fix: **lower max_depth** or **lower learning_rate**.
- Trees and forests need **no scaling**.

Code 9. Support Vector Machines

Linear SVM and RBF kernel SVM

```

from sklearn.svm import LinearSVC
linear_svm = LinearSVC().fit(X, y)

from sklearn.svm import SVC
svm = SVC(kernel='rbf', C=10, gamma=0.1).fit(X, y)

svc = SVC()
svc.fit(X_train, y_train)
print("Accuracy on training set: {:.2f}".format(svc.score(X_train, y_train)))
print("Accuracy on test set: {:.2f}".format(svc.score(X_test, y_test)))

```

Scaling by hand to 0-1 (the slides do NOT use MinMaxScaler here)

```

# compute the minimum value per feature on the training set
min_on_training = X_train.min(axis=0)
# compute the range of each feature (max - min) on the training set
range_on_training = (X_train - min_on_training).max(axis=0)

# subtract the min, and divide by range
# afterward, min=0 and max=1 for each feature
X_train_scaled = (X_train - min_on_training) / range_on_training

# use THE SAME transformation on the test set,
# using min and range of the training set
X_test_scaled = (X_test - min_on_training) / range_on_training

svc = SVC()
svc.fit(X_train_scaled, y_train)

```

```
print("Accuracy on training set: {:.3f}".format(svc.score(X_train_scaled,
y_train)))
print("Accuracy on test set: {:.3f}".format(svc.score(X_test_scaled, y_test)))

svc = SVC(C=1000)
svc.fit(X_train_scaled, y_train)
```

- Default SVC on the raw breast cancer data overfits (train **1.00**, test **0.63**). **Scaled**, default SVC gives train **0.948**, test **0.951**. **C=1000** on scaled data gives train **0.988**, test **0.972**.
- The test set must use the **training set's** min and range, never its own (that would leak information).
- **MinMaxScaler** from sklearn.preprocessing does the same job and is what the M1 notebook uses.

Watch Out

Slide naming: k-NN is **clf**, the regressor is **reg**, ridge is **ridge**, the forest is **forest**, boosting is **gbdt**, and the SVM is **svc**. Every model follows the same pattern: create, **fit(X_train, y_train)**, then **score** on train and on test. Classifiers report accuracy, regressors report R^2 .

Cram Sheet

Everything on one page. **SA1 items first**, midterm-only items after the line.

- **ML** = extracting knowledge from data; learns from **data / experience** to predict with minimal human intervention. **Samuel, 1959. AI contains ML contains Deep Learning.**
- **E, T, P:** performance at task T, measured by P, improves with experience E.
- **Rule-based = condition**, manually specified rules; **ML = data**, rules learned automatically.
- **Four types** (picture): supervised, unsupervised, **semi-supervised**, reinforcement. Workflow: **project setup, data preparation, modeling, deployment.**
- Python is **#1** (then R, Java). Libraries: **pandas, NumPy, SciPy, Matplotlib, Seaborn**, scikit-learn, TensorFlow, Keras, PyTorch.
- Data problems: **incompleteness, noise, inconsistency**. Preprocessing steps: **profiling, cleansing, reduction, transformation, enrichment, validation**. Data scientists spend **60 percent** on cleaning.
- **Outlier** = far from the rest; **outlier detection** = EDA process. Missing values: **remove rows or impute** (mean, most frequent).
- **Discrete** = countable. **Continuous** = unbroken scale. **Classification** = categorical target. **Regression** = continuous target.
- **Generalization** = good on unseen data. **Underfitting** = too simple, **high bias**. **Overfitting** = memorized training, **high variance**. **Low bias + low variance = good model.**
- **Small k** = complex = overfit. **Large k** = simple = underfit. k-NN: store data, vote (class) or average (regression), Euclidean by default.
- **R2** = $1 - SS_{res} / SS_{tot}$: 1 perfect, 0 = predicts the mean, **negative = worse than the mean.**
- **y-hat** = $w*x + b$; **b** = intercept = offset = bias. **OLS** minimizes mean squared error on the training set.
- OLS: small gap and mediocre score = underfit; huge train-test gap (many features) = overfit.
- **Code:** load, **train_test_split** (X_train, X_test, y_train, y_test), **KNeighborsClassifier(n_neighbors=k)**, **fit(train)**, **predict(test)**, **score(test)**. **SimpleImputer(strategy='mean')** and **StandardScaler / MinMaxScaler** use **fit_transform(df[['col']])**.
- ---- SA1 ends at linear regression. Midterm-only below. ----
- **Ridge** = L2, cost MSE + $\alpha * \sum(w \text{ squared})$, never exactly zero; **larger alpha = larger penalty**. **Lasso** = L1, some weights **exactly zero**.
- **Logistic Regression** = classification (sigmoid, threshold). **C high = less regularization**.
- **Naive Bayes:** each feature alone, per-class stats. **Gaussian** continuous, **Multinomial** counts, **Bernoulli** binary.
- **Decision tree** = if/else tests, cannot extrapolate. **Random forest** = average of independent bootstrapped trees. **Gradient boosting** = shallow trees fixing the previous error, learning rate 0.1.

- **SVM** = kernel trick for nonlinear data; **scale to 0-1**; small gamma = simpler; small C = restricted.